LA (UE23MA241B) Hackathon-2

Smart City Traffic Flow Optimization

Members:

Rithvik Rajesh Matta **PES2UG23CS485**

Rishi A Sheth **PES2UG23CS479**

Code:

```
import numpy as np
from scipy.linalg import lu_factor, lu_solve

def transform_data(X, threshold=0.95):
    """
    Reduce the dimensionality of the high-dimensional sensor data X.

    Parameters:
    - X: np.ndarray; a matrix where each row corresponds to an
intersection's measurements.
    - threshold: float; the cumulative contribution to the total variance
to retain.

    Returns:
    - X_reduced: np.ndarray; the transformed data capturing the essential
features.
    """
    # Center the data
    x_centre = X - np.mean(X, axis=0)

    #covariance matrix
    cov_matrix = np.cov(x_centre, rowvar=False)

    #e_val and e_vec
    e_val, e_vec = np.linalg.eigh(cov_matrix)
```

```
    #descending order
    idx = np.argsort(e_val)[::-1]
    e_val = e_val[idx]
    e_vec = e_vec[:, idx]

    #cumulative variance
    total_var = np.sum(e_val)
    explained_var_r = e_val / total_var
    cum_var_ratio = np.cumsum(explained_var_r)

    #number of components to keep
    n_components = np.argmax(cum_var_ratio >= threshold) + 1

    #necessary e_vec
    p_comp = e_vec[:, :n_components]

    # Projecting the data
    X_reduced = x_centre @ p_comp

    for i in range(X_reduced.shape[1]):
        first_nonzero_idx = np.nonzero(X_reduced[:, i])[0]
        if len(first_nonzero_idx) > 0 and X_reduced[first_nonzero_idx[0],
i] > 0:
            X_reduced[:, i] = -X_reduced[:, i]

    return X_reduced

def construct_equations(transformed_data, params):
    """
    Construct the linear system A*x = b based on the transformed data and a
parameter vector.

    Parameters:
    - transformed_data: np.ndarray; the output from transform_data.
    - params: np.ndarray; a vector of parameters whose length equals the
effective dimension (number of columns of transformed_data).

    Returns:
    - A: np.ndarray; the coefficient matrix.
```

```
    - b: np.ndarray; the right-hand side vector.
    """
    #column vector
    if len(params.shape) == 1:
        params = params.reshape(-1, 1)

    #calculating contribution to A and b
    A = transformed_data.T @ transformed_data
    b = transformed_data.T @ (transformed_data @ params)

    return A, b

def compute_adjustments(A, b, tol=1e-10):
    """
    Solve the system A*x = b to compute the adjustments x.

    If A is ill-conditioned, handle accordingly.
    """
    #ill-conditioned check
    cond_no = np.linalg.cond(A)

    if cond_no > 1/tol:
        #SVD
        u, s, vh = np.linalg.svd(A)

        #threshold
        s_inv = np.zeros_like(s)
        s_inv[s > tol * s[0]] = 1 / s[s > tol * s[0]]

        #pseudo-inverse
        A_pinv = vh.T @ np.diag(s_inv) @ u.T
        x = A_pinv @ b
    else:
        #LU decomposition
        lu, piv = lu_factor(A)
        x = lu_solve((lu, piv), b)

    return x

def evaluate_solution(A, x, b):
```

```
    """
    Compute the Euclidean norm ||Ax - b||_2.
    """
    residual = np.linalg.norm(A @ x - b)
    return residual

def main():
    ##DO NOT CHANGE CODE HERE
    sensor_data_list = []
    while True:
        try:
            row = input().strip()
            if row:
                sensor_data_list.append(list(map(float, row.split())))
            else:
                break
        except EOFError:
            break
    sensor_data = np.array(sensor_data_list)

    param_line = ""
    while True:
        try:
            param_line = input().strip()
            if param_line:
                break
        except EOFError:
            break
    params = np.array(list(map(float, param_line.split())))

    transformed = transform_data(sensor_data)
    A_mat, b_vec = construct_equations(transformed, params)
    x = compute_adjustments(A_mat, b_vec)
    residual = evaluate_solution(A_mat, x, b_vec)

    print("Transformed Data:")
    print(np.array2string(transformed, precision=2, suppress_small=True))
    print("Matrix A:")
    print(np.array2string(A_mat, precision=2, suppress_small=True))
    print("Vector b:")
```

```
    print(np.array2string(b_vec, precision=2, suppress_small=True))
    print("Solution x:")
    print(np.array2string(x, precision=2, suppress_small=True))
    print("Residual norm:")
    print(f"{residual:.2f}")


if __name__ == "__main__":

    main()
```

Output:

Test case 1

```
rithvikmatta@penguin:~/sem4/LAA/datathon 2$ python3 Q2_PES2UG23CS485_PES2UG23CS479.py
10 12 14
20 22 24
30 32 34

1
Transformed Data:
[[-17.32]
 [  0.  ]
 [ 17.32]]
Matrix A:
[[600.]]
Vector b:
[[600.]]
Solution x:
[[1.]]
Residual norm:
0.00
```

Test case 2

```
rithvikmatta@penguin:~/sem4/LAA/datathon 2$ python3 Q2_PES2UG23CS485_PES2UG23CS479.py
1 2 3
4 5 6
7 8 10
9 10 13

0.5
Transformed Data:
[[-7.81]
 [-2.65]
 [ 3.18]
 [ 7.29]]
Matrix A:
[[131.26]]
Vector b:
[[65.63]]
Solution x:
[[0.5]]
Residual norm:
0.00
```

Explanation of problem statement:

In this project, we are using traffic sensor data from a smart city to enhance the manner in which traffic lights are modified. The challenge is that the data is too big, noisy, and repeats itself in a similar pattern frequently, making it difficult to directly analyze. What we want to do is to clean and summarize the data down to the most critical features that actually influence traffic. Next, we will construct a mathematical model (in the form of a linear system) to see how traffic responds and how we can manipulate signals to mitigate congestion. Lastly, we will ensure our model and findings are reasonable by checking the stability and consistency of the solution.

Solution Overview:

We tackle this problem through a structured four-step process:

1. Dimensionality Reduction (transform_data)

We employ Principal Component Analysis (PCA) to distill the original high-dimensional sensor data into its most impactful features.

2. System Formulation (construct_equations)

After obtaining the transformed data, we represent the linear system in the format $A \cdot x = b$:

Here, A encapsulates the relationships among the transformed features, while b is derived by projecting expected outputs based on parameters into the transformed space. This approach simulates the influence of various traffic features on signal timing decisions.

3. System Resolution (compute_adjustments)

Depending on the characteristics of matrix A, we resolve the system using:

LU decomposition for well-conditioned (stable) matrices, and SVD-based pseudoinverse for matrices that are near-singular, thereby avoiding unstable or erroneous outcomes. This process ensures that we derive a dependable solution x, which reflects the optimal adjustments for traffic signals.

4. Model Assessment (evaluate_solution)

We calculate the residual norm $||Ax - b||_2$ to evaluate the fit of our solution within the system. A smaller residual indicates a superior fit, suggesting that the adjustments are coherent within the model's context.

Output Explanation:

Transformed Data:

This represents the reduced-dimensional version of the original sensor data, emphasizing the critical traffic features at intersections.

Matrix A:
This coefficient matrix of the linear system is derived from the transformed data, illustrating how each principal feature affects the system.

Vector b:
This component of the system signifies the anticipated traffic signal adjustments based on parameters set by city planners.

Solution x:
This adjustment vector is obtained by solving $A \cdot x = b$, with each element in x reflecting the contribution of a principal feature to the overall changes in traffic signals.

Residual Norm:
This is the Euclidean norm $||Ax - b||_2$, which serves to confirm the accuracy and stability of the solution. A low value indicates that the model is consistent and reliable.